US9: External API Integration Plan

Chronicles Development Team

1 US9: External API Integration Plan

1.1 Objective

The objective of US9 is to research and select an external API that can provide useful functionality for the Chronicles digital logbook and to document the proposed integration approach before implementation begins.

This user story is focused on planning and documentation only. No external API integration is implemented as part of this user story.

1.2 Candidate APIs

The team considered several external APIs that could potentially extend the functionality of the Chronicles digital logbook:

- **Google Calendar API**: Could be used to synchronise project deadlines, scheduled events, and other time-based activities with a user's calendar.
- **OpenWeather API**: Could provide weather information for field-based, outdoor, environmental, or other location-dependent logbook projects.
- **Notification/Email API**: Could be used to provide reminders and notifications related to projects, deadlines, or logbook entries.

1.3 Selected API

The team selected the **OpenWeather API** as the external API for the planned integration.

1.4 Rationale

OpenWeather was selected because weather information can provide useful context for certain types of logbook projects. Chronicles is designed to support customisable projects, meaning that users may create projects involving fieldwork, outdoor activities, environmental observations, or other location-dependent activities.

Weather information could therefore be associated with a logbook entry to provide additional context about the conditions under which the entry was recorded.

The OpenWeather API also provides a REST-based interface, making it suitable for integration with the existing FastAPI backend. This allows the team to introduce the external service without requiring a major change to the current system architecture.

Another reason for selecting OpenWeather is that the integration can remain optional. Projects that do not require environmental or weather information can continue to use the existing Chronicles functionality without depending on the external service.

1.5 Proposed Integration Approach

The external API will be integrated through the Chronicles backend. The React frontend will communicate only with the Chronicles API, while the FastAPI backend will communicate with the OpenWeather API.

The proposed integration flow is:

1. The user creates or views a logbook entry associated with location information.
2. The Chronicles backend obtains the relevant location information from the project or entry.
3. The backend sends a request to the OpenWeather API using the relevant location information.
4. OpenWeather returns the available weather information.
5. The Chronicles backend processes the response and extracts the relevant weather data.
6. The weather information is associated with the relevant logbook entry or displayed as additional project context.
7. The React frontend displays the weather information where appropriate.

Keeping communication with OpenWeather within the backend ensures that API credentials are not exposed to the frontend client. It also provides a single location for handling external API requests, validation, error handling, and configuration.

1.6 Configuration Requirements

The eventual implementation will require an API key obtained from the external weather service. The API key must not be committed to the source-code repository.

Instead, the API credentials and related configuration will be stored using environment variables. The eventual implementation may require configuration similar to:

```
OPENWEATHER_API_KEY=<API key>
OPENWEATHER_BASE_URL=<OpenWeather API base URL>
```

The acquisition of the API key and configuration of these environment variables will be handled as follow-up implementation work.

1.7 Potential Implementation Considerations

Before implementation, the team will need to consider several factors:

- **API availability**: The system should handle cases where the external weather service is unavailable.
- **Invalid locations**: Invalid or incomplete location information should not cause the entire logbook operation to fail.
- **API limits**: The implementation should account for request limits imposed by the external service.
- **Error handling**: Failed requests and unexpected responses should be handled gracefully by the backend.
- **Credential security**: API keys must be stored in environment configuration and must not be exposed in frontend code or committed to version control.
- **Optional functionality**: Weather information should remain supplementary so that users can continue using Chronicles when weather data is unavailable or irrelevant to their project.

1.8 Implementation Scope

US9 is limited to researching and documenting the selection of an external API and the proposed integration approach.

The following are explicitly outside the scope of US9:

- Implementing external API requests.
- Adding API credentials to the application.
- Making database schema changes.
- Adding weather-related frontend functionality.
- Persisting weather data in logbook entries.

The actual OpenWeather integration will be implemented in a subsequent user story.

1.9 Acceptance Criteria

The requirements of US9 are satisfied by documenting:

1. The external API selected by the team.
2. The rationale for selecting the API.
3. A rough integration plan describing how the frontend, Chronicles backend, and external API will communicate.
4. The configuration requirements that will be needed for the future implementation.
5. The fact that the current user story does not include implementation of the external API.